

SQL Notes – Joins, Aggregate Functions, Primary Keys & Foreign Keys

1. Aggregate Functions in SQL

Aggregate functions perform calculations on multiple rows and return a single value.

Function	Purpose
COUNT()	Counts rows
SUM()	Adds values
AVG()	Finds average
MAX()	Finds highest value
MIN()	Finds lowest value

COUNT()

```
SELECT COUNT(*)  
FROM employees;
```

Counts total employees.

SUM()

```
SELECT SUM(salary)  
FROM employees;
```

Returns total salary.

AVG()

```
SELECT AVG(salary)  
FROM employees;
```

Returns average salary.

MAX()

```
SELECT MAX(salary)
FROM employees;
```

Returns highest salary.

MIN()

```
SELECT MIN(salary)
FROM employees;
```

Returns lowest salary.

GROUP BY Example

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id;
```

HAVING Example

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

2. SQL JOINS

Assume two tables:

employees

emp_id	name	dept_id
1	John	10
2	Sara	20
3	Alan	30

departments

dept_id	dept_name
10	HR
20	Finance
40	Sales

INNER JOIN

Returns matching rows only.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

LEFT JOIN

Returns all rows from left table.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

RIGHT JOIN

Returns all rows from right table.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

FULL OUTER JOIN (MySQL Alternative)

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id

UNION

SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

CROSS JOIN

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

SELF JOIN

```
SELECT e.name AS Employee,
       m.name AS Manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

USING()

```
SELECT name, dept_name
FROM employees
JOIN departments
USING(dept_id);
```

NATURAL JOIN

```
SELECT *
FROM employees
NATURAL JOIN departments;
```

3. Primary Keys & Foreign Keys

Creating Table with Primary Key

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

Adding Primary Key

```
ALTER TABLE students
ADD PRIMARY KEY(student_id);
```

Dropping Primary Key

```
ALTER TABLE students  
DROP PRIMARY KEY;
```

Creating Foreign Key

Parent Table

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

Child Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY(dept_id)  
    REFERENCES departments(dept_id)  
);
```

Finding Foreign Key Name

```
SHOW CREATE TABLE employees;
```

Dropping Foreign Key

```
ALTER TABLE employees  
DROP FOREIGN KEY employees_ibfk_1;
```

Dropping Referenced Column

```
ALTER TABLE employees  
DROP COLUMN dept_id;
```

Changing Primary Key

Remove old primary key

```
ALTER TABLE employees  
DROP PRIMARY KEY;
```

Add new primary key

```
ALTER TABLE employees  
ADD PRIMARY KEY(email);
```

Composite Primary Key

```
CREATE TABLE orders (  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    PRIMARY KEY(order_id, product_id)  
);
```

UNIQUE Constraint

```
ALTER TABLE employees  
ADD UNIQUE(email);
```

NOT NULL Constraint

```
ALTER TABLE employees  
MODIFY salary DECIMAL(10,2) NOT NULL;
```

4. Practice Questions

Aggregate Functions

1. Find total salary of employees.
 2. Find average salary department-wise.
 3. Find highest salary.
 4. Find lowest salary in HR department.
 5. Count employees in each department.
-

JOIN Questions

1. Display employee names with department names using INNER JOIN.
 2. Show all employees even if department is missing.
 3. Show all departments even if no employees exist.
 4. Find employees and managers using SELF JOIN.
 5. Display all employee-department combinations using CROSS JOIN.
-

Constraints & Keys

1. Create a table with primary key.
 2. Add a primary key to existing table.
 3. Drop primary key.
 4. Create foreign key relationship.
 5. Drop foreign key constraint.
 6. Create composite primary key.
 7. Add UNIQUE constraint.
 8. Add NOT NULL constraint.
 9. Remove referenced column.
 10. Change primary key.
-

5. Useful Commands

Describe Table

```
DESCRIBE employees;
```

Show Table Creation Query

```
SHOW CREATE TABLE employees;
```

Explain Query

```
EXPLAIN SELECT * FROM employees;
```

Quick Summary

Concept	Purpose
PRIMARY KEY	Unique identifier
FOREIGN KEY	Creates relationship
INNER JOIN	Matching rows only
LEFT JOIN	All left rows
RIGHT JOIN	All right rows
COUNT()	Counts rows
SUM()	Adds values
AVG()	Average value
GROUP BY	Groups rows
HAVING	Filters grouped data